

Integração: Abertura de Conta Escrow (com Risk Solution)

Visão geral

Este documento descreve o fluxo de integração para abertura de contas Escrow, considerando que os dados passam pelo serviço de Risk Solution antes da reserva/abertura final na plataforma QiTech/PlugQi.

Fluxo resumido:

1. Enviar dados ao Risk Solution (Natural ou Legal).
2. Aguardar análise (automatic_approval / manual_analysis / automatic_rejection).
3. Se aprovado/pendente, criar reserva de conta Escrow (POST /account_request/escrow).
4. Aguardar webhook de atualização de status (ex.: pending_additional_data, pending_bacen_validation).
5. Enviar confirmação (PATCH /account_request/{account_request_key}/escrow) com dados completos.

Requisitos

- Chave de API para os endpoints de Risk Solution.
 - Cliente HTTP autenticado para os endpoints da PlugQi.
 - Listener para webhooks (opcional em ambiente de testes; o fluxo pode ser simulado).
-

Risk Solution

A integração com Risk Solution é feita através dos endpoints:

- POST {base_url}/natural_person?analyze=true — para PF (CPF formatado: ###.###.###-##).
- POST {base_url}/legal_person?analyze=true — para PJ (CNPJ formatado: ##.###.###/####-##).

Headers:

- Authorization: <API_KEY>
- Content-Type: application/json

Exemplo de request (Natural Person - PF):

```
{
  "document_number": "123.456.789-10",
  "name": "Fulano da Silva",
  "birth_date": "1990-05-06",
  "email": "fulano@example.com",
  "phone": {
    "country_code": "055",
    "area_code": "11",
    "number": "999999999"
  }
}
```

```
}  
}
```

Exemplo de request (Legal Person - PJ):

```
{  
  "document_number": "12.345.678/0001-90",  
  "name": "Empresa Exemplo LTDA",  
  "foundation_date": "2010-01-01",  
  "email": "contato@empresa.com",  
  "phone": {  
    "country_code": "055",  
    "area_code": "11",  
    "number": "12345678"  
  }  
}
```

Resposta típica (exemplo simplificado):

```
{  
  "id": "rs_abc123",  
  "status": "automatic_approval",  
  "score": 85,  
  "details": {  
    "checks": []  
  }  
}
```

Observações:

- Se o Risk Solution retornar **automatic_rejection** interrompa o fluxo e retorne erro ao seu cliente.
- Se retornar **manual_analysis**, sinalize que será necessário acompanhamento humano e continue conforme regras internas.

Endpoints de Escrow (PlugQi)

Base: <https://<plugqi-host>/> (use sandbox/produção conforme ambiente)

1. Criar reserva (POST)

- Endpoint: **POST /account_request/escrow**
- Uso: iniciar a solicitação de abertura (PF ou PJ). Retorna **account_request_key** e **account_request_status**.

Exemplo (PF) - Request:

```
{
  "account_owner": {
    "document_number": "99999999999",
    "email": "teste@gmail.com",
    "birthdate": "1990-05-06",
    "name": "Nome do Titular da Conta",
    "documents": {
      "cnh": { "ocr_key": "<uuid>" }
    },
    "face": "<uuid>"
  }
}
```

Exemplo (Resposta de Reserva):

```
{
  "account_info": {
    "account_branch": "0001",
    "account_digit": "2",
    "account_number": "2797627"
  },
  "account_request_key": "fee89de6-e62e-4e88-b77d-457ebb439555",
  "account_request_status": "pending_bacen_validation"
}
```

2. Consulta de status (GET)

- Endpoint: **GET** /account_request/{account_request_key}
- Uso: consultar o status atual da solicitação de abertura.

3. Confirmação da abertura (PATCH)

- Endpoint: **PATCH** /account_request/{account_request_key}/escrow
- Uso: enviar dados completos (endereço, assinatura, documento de prova de residência, destinos, etc.) para concluir a abertura.

Exemplo (PATCH - confirmação completa):

```
{
  "account_owner": {
    "address": {
      "street": "Av. Brigadeiro Faria Lima",
      "state": "SP",
      "city": "São Paulo",
      "neighborhood": "Jardim Paulistano",
      "number": "2391",
      "postal_code": "01452905",
      "complement": "Complemento"
    }
  }
}
```

```

    },
    "birth_date": "1990-05-06",
    "document_identification": "<uuid>",
    "email": "teste@gmail.com",
    "individual_document_number": "9999999999",
    "is_pep": false,
    "mother_name": "Dona Maria Mariane",
    "name": "Nome do Titular da Conta",
    "nationality": "Brasileira",
    "person_type": "natural",
    "phone": {
      "country_code": "055",
      "area_code": "11",
      "number": "999999999"
    },
    "proof_of_residence": "<uuid>"
  },
  "signed_contract": {
    "document_key": "<uuid>",
    "signatures": [
      {
        "authenticity": {
          "timestamp": "2025-01-01T12:00:00Z",
          "facial_recognition_key": "<uuid>",
          "lang": "-35.8916627",
          "lat": "-7.2226067",
          "ip_address": "177.51.1.186",
          "session_id": "jdifj329842"
        },
        "signer": {
          "name": "Nome do Titular da Conta",
          "email": "teste@gmail.com",
          "phone": {
            "country_code": "055",
            "area_code": "11",
            "number": "999999999"
          },
          "document_number": "9999999999"
        },
        "authentication_type": "opt-in"
      }
    ]
  },
  "destinations": [
    {
      "account_branch": "0001",
      "account_number": "1234567",
      "account_digit": "1",
      "document_number": "04252012000123",
      "name": "Conta do FIDC",
      "ispb_number": "32402502",
      "financial_institution_code_number": "329"
    }
  ],

```

```
"additional_documents": ["<uuid>"]
}
```

Resposta esperada (sucesso):

```
{
  "account_key": "acc_abcdef123456",
  "status": "active",
  "created_at": "2025-01-01T12:00:00Z"
}
```

Erros comuns durante PATCH:

- Código **ACR000042**: conta ainda não está pronta para confirmação (aguardar webhook com status **pending_additional_data**).

Webhooks

A QiTech envia webhooks de atualização de status sobre a **account_request**. Exemplo (simulado no repositório):

Webhook example (payload):

```
{
  "key": "fee89de6-e62e-4e88-b77d-457ebb439555",
  "data": {
    "account_info": {
      "account_digit": "7",
      "account_branch": "0001",
      "account_number": "6694401"
    },
    "account_request_key": "fee89de6-e62e-4e88-b77d-457ebb439555"
  },
  "status": "pending_additional_data",
  "webhook_type": "account_request.status_change",
  "event_datetime": "2025-01-01T12:00:00"
}
```

Observações para o listener:

- Validate signature if provided (considere HMAC if a signature key for webhooks estiver configurada).
- Ao receber **pending_additional_data** indicar ao cliente que envie a etapa de confirmação (PATCH) com dados completos.

Exemplo de fluxo completo (resumido)

1. Envie payload para Risk Solution (/natural_person ou /legal_person).
2. Se automatic_rejection — pare o fluxo e notifique.
3. Se automatic_approval ou manual_analysis — prossiga com POST /account_request/escrow.
4. Salve account_request_key retornado.
5. Aguardar webhook da QiTech (ou consultar com GET /account_request/{account_request_key}).
6. Quando status permitir, enviar PATCH /account_request/{account_request_key}/escrow com dados completos.

Apêndice — Exemplos extraídos do repositório

- Script de reserva (exemplo PF): escrow_complete_flow.py — payload usado na reserva inicial.
- Exemplo de resposta de reserva: escrow_pj_response_new.json (contido no repositório).

Observações finais

- Garanta que os formatos de CPF/CNPJ estejam corretos antes de chamar Risk Solution.
- Mantenha logs das chamadas ao Risk Solution e aos endpoints de Open Account para auditoria.
- Para testes, use as rotas sandbox e simule webhooks quando necessário.

Se desejar, posso:

- Gerar um PDF deste documento automaticamente (requer instalar dependências Python).
- Gerar um README mais curto ou uma versão em inglês.

Apêndice B — Exemplos práticos e verificação

Exemplos CURL — Risk Solution

PF (Natural Person):

```
curl -X POST "https://api.sandbox.caas.qitech.app/onboarding/natural_person?analyze=true" \
-H "Authorization: <API_KEY>" \
-H "Content-Type: application/json" \
-d '{
  "document_number": "123.456.789-10",
  "name": "Fulano da Silva",
  "birth_date": "1990-05-06",
  "email": "fulano@example.com"
}'
```

PJ (Legal Person):

```
curl -X POST "https://api.sandbox.caas.qitech.app/onboarding/legal_person?analyze=true" \
```

```
-H "Authorization: <API_KEY>" \  
-H "Content-Type: application/json" \  
-d '{  
  "document_number": "12.345.678/0001-90",  
  "name": "Empresa Exemplo LTDA",  
  "foundation_date": "2010-01-01",  
  "email": "contato@empresa.com"  
}'
```

Exemplos CURL — PlugQi Escrow

Criar reserva (POST):

```
curl -X POST "https://{plugqi-host}/account_request/escrow" \  
-H "Content-Type: application/json" \  
-d '{  
  "account_owner": { "document_number": "999999999999", "email":  
"teste@gmail.com", "birthdate": "1990-05-06", "name": "Nome do Titular da Conta" }  
}'
```

Consultar status (GET):

```
curl -X GET "https://{plugqi-host}/account_request/{account_request_key}" \  
-H "Content-Type: application/json"
```

Confirmar abertura (PATCH):

```
curl -X PATCH "https://{plugqi-host}/account_request/{account_request_key}/escrow"  
\  
-H "Content-Type: application/json" \  
-d '{ /* payload de confirmação completo */ }'
```

Respostas de erro comuns

Exemplo — conta não pronta (ACR000042):

```
{  
  "error": "Account not ready",  
  "code": "ACR000042",  
  "message": "Account request still in status pending_additional_data"  
}
```

Exemplo — rejeição automática no Risk Solution:

```
{
  "id": "rs_abc123",
  "status": "automatic_rejection",
  "score": 12,
  "details": { "reason": "high_risk_document" }
}
```

Verificação de assinatura HMAC (exemplo Python)

O repositório contém a função `compute_webhook_signature(endpoint, method, payload, signature_key)` em `risk_solution.py`.

Exemplo de verificação para uso no listener:

```
import hmac
import hashlib

def verify_webhook(endpoint: str, method: str, payload_str: str, header_signature: str, signature_key: str) -> bool:
    message = (endpoint + method + payload_str).encode('utf-8')
    expected = hmac.new(signature_key.encode('utf-8'), message,
        hashlib.sha1).hexdigest()
    return hmac.compare_digest(expected, header_signature)

# uso:
# verify_webhook('/webhook/escrow', 'POST', received_body_string,
#     header_signature_value, 'your_signature_key')
```

Se quiser que eu gere o PDF automaticamente, autorize instalar dependências (`markdown`, `weasyprint` ou `pandoc`) ou me diga qual ferramenta prefere.